

1. **ISCED Code:** 0613
2. **Course Code:** CSE-3527
3. **Course Title:** Compiler
4. **Type:** Core, Engineering
5. **Semester:** 5th
6. **Credit Hours:** 3
7. **Contact Hours:** 3 lecture hours per week
8. **CIE Marks :** 50
9. **SEE Marks:** 50
10. **Total marks:** 100
11. **Prerequisite:** CSE-2425 (Theory of Computing)
12. **Co-requisite:**

1. **ISCED Code:** 0613
2. **Course Code:** CSE-3527
3. **Course Title:** Compiler
4. **Credit Hours:** 3
5. **Contact Hours:** 3 lecture hours per week
6. **Type:** Core, Engineering
7. **Prerequisite:** CSE-2425 Theory of Computing
8. **Instructor's and Class Schedule and Locations**

Instructor: Israt Binte Habib

Office Location: Room # 307, Female CSE building

Email: israthabib.cse@iiuc.ac.bd

9. Course Rationale / Summary:

This is a course to familiarize the students with knowledge and ideas of the construction of computer program translators called compilers. To take this course, students should be able to gain the skills of automata theory which need to design and implementation of the compiler. The competitive and standard programming ability with some mathematical maturity also will be expected; students should have some idea of computer language grammar and computing skills will also be helpful. After studying this course, the students will have in-depth knowledge of the design and implementation of compilers maintaining the standard steps.

8. Course Objective: The objectives of this course are to

1. Grasp of compiler construction.
2. To introduce the major concept areas of language translation and compiler design
3. To enrich the knowledge in various phases of compiler and its use, code optimization techniques, machine code generation, and use of symbol table.
4. To extend the knowledge of parser and understand the theory and practice of compiler implementation.
5. To provide practical programming skills necessary for constructing a compiler
6. Utilize tools to automate compiler construction.
7. Develop a large, complex, but well-structured software system that implements various phases of a compiler such as the scanner, parser, code generator, and optimizer.
8. Identify the similarities and differences among various parsing techniques and grammar transformation techniques

9. Knowledge Profile (WK):

WK3	Engineering fundamentals	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline
WK4	Specialist knowledge	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK6	Engineering practice	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline

10. Complex Engineering Problem Solve: P1

11. Complex Engineering Activities: N/A

12. Learning Domains:

Cognitive - C2: Understanding, C3: Applying, C4: Analyzing, C5: Evaluating, C6: Create

Psychomotor - P2: Manipulation, P3: Precision

Affective - A2: Responding

13. Course Outcomes (COs):

Upon successful completion of this course, students will be able to:

#	CO Description	Weightage (%)
CO1	Understand the modern compiler construction concepts for advanced computer languages	10%
CO2	Apply the theory of computation for compiler construction	20%
CO3	Analyze the lexical, syntax, semantic phases and parser of front end compiler construction	20%
CO4	Evaluate the different techniques for code generation and code optimization of back-end compiler construction	40%
CO5	Ability to design a well-structured compiler that implements various compiler phases, such as the scanner, parser, code generator, and optimizer	10%

14. Mapping of CO-PO-WK-WP-WA:

#	CO Description	POs	DL	KP	EP	EA	TLS	AS
CO1	Understand the modern compiler construction concepts for advanced computer languages	PO1	C2	WK3			Lecture, Class discussion, Note	Exam, Quiz, Assignment
CO2	Apply the theory of computation for compiler construction	PO1	C3	WK3			Discussion, Assignment	Assignment, Quiz, Exam
CO3	Analyze the lexical, syntax, semantic phases and parser of front-end compiler construction	PO2	C4	WK3			Lecture, Assignment, Quiz	CT, Assignment
CO4	Evaluate the different techniques for code generation and code optimization of back-end compiler construction	PO5	C5	WK4, WK6	P1			CT, Assignment
CO5	Ability to design a well-structured compiler that implements various compiler phases, such as the scanner, parser, code generator, and optimizer	PO2	C6	WK4	P1			Exam, Assignment
Note: DL: Domain/level of learning taxonomy, KP: Knowledge Profile, EP: Attribute of Complex Engineering Problems, EA: Attribute of Complex Engineering Activities Learning Domains (C: Cognitive, A: Affective, P: Psychomotor)								

15. Weightage Distribution among Assessment Tools:

Assessment Tools	Weightage (%)	
Class Attendance	10	100
Class test and Assignments	10	
Mid-Term Exam	30	
Final Exam	20 (Group A)	
	30 (Group B)	50

16. Grading Policy: As per IIUC grading policy

17. Course Content:

#	Content	Duration	CLOs
Section-A (Mid-term: 30 Marks)			
1	Segment 1: Structure of compiler: Translator, Basic principles of compilers and compiler design, types of compilers, applications, phases of a compiler.	2 weeks	CLO1
2	Segment 2: Grammars & expressions: Concepts of languages and Grammars, Terminals and nonterminal, Useless non-terminals and Inaccessible tokens, Context-Sensitive Grammars, Context-Free Grammars, right linear grammar, left linear grammar, Derivations and Parse trees, Ambiguous grammar.		
3	Segment 3: Automata: Lexical analysis, Regular expressions, finite automata, Non-deterministic Finite State Automata, Minimization of Finite-State Automata, DFA, Symbol table management		
Section-B (Final Exam: 50 Marks)			
Group-A (20 Marks)			
4	Segment 4: Syntax Analysis: Parser, Error-Recovery, Left recursion, Left factoring, Top-Down Parsers, FIRST and FOLLOW, Recursive-Descent and Predictive Parsing, LL(1) parsing.		
5	Segment 5: Parsing: Bottom-Up Parsing, Operator Precedence parsing, LR Parsers, Construction of SLR, CLR, LALR Parse Tables, Shift-Reduce Parsing.		
Group-B (30 Marks)			
6	Segment 6: Syntax-Directed Translation: Syntax-Directed definitions, Semantic Rules, Dependency Graph, Attributes, Construction of Syntax trees, Type checking, Type expressions, Translation Scheme, Type checking of expressions, statements and functions, Type conversions.		
7	Segment 7: Intermediate code generation: Run time environments, Activation tree, Control stack, run time storage organization, Storage allocation strategies, Intermediate languages, three address statements, quadruples, triples, indirect triples and polish notations.		
8	Segment 8: Code Generation and Optimization: Code generation issues, Instruction costs, Basic blocks, Flow graphs, DAG representation, Next use information, Transformations of Basic Blocks, Peephole optimization, Function preserving transformations, Optimization of Basic Blocks, Different models of code generation, error detection and error handling.		

18. Weekly Activity Plan:

Activities: Lectures (L), Class Test (CT), Assignment (A)

Week	Activities				
	Lectures Discussion Topics Segments Assigned				
1	L1	Introduction: Class information and Introduction to compiler	Lecture	CT, Mid	CLO1
1	L2, L3	Structure of compiler: Translator, Basic principles of compilers and compiler design	Lecture, Discussion	Class work, Assignment	CLO1
2	L4, L5	Structure of compiler: types of compilers, applications, phases of a compiler	Lecture, Discussion	Class work, Quiz	CLO2
2	L6	Review of Segment 1 Class Test – 1	Lecture, Discussion	Class work, CT, Quiz, Assignment	CLO2
3	L7, L8	Grammars and Expressions: Concepts of languages and Grammars, Terminals and non-terminals	Lecture, Discussion	CT, Assignment	CLO2
3	L9	Grammars and Expressions: Useless non-terminals and Inaccessible tokens			
4	L10	Grammars & Expressions: Context-Sensitive Grammars, right linear grammar, left linear grammar			
4	L11, L12	Grammars & Expressions: Context-Free Grammars Derivations and Parse trees, Ambiguous grammar Review of Segment 2	S-2	A-1	
5	L13	Automata: Lexical analysis, Regular expressions	S-3		
5	L14, L15	finite automata, deterministic Finite-State Automata(DFA), non-deterministic Finite-State Automata (NFA)			

6	L16	Automata: Minimization of Finite-State Automata, Symbol table management			
6	L17	Review of segment 3 Class Test – 2	S-3	CT-2	
6	L18	Discussion Class for Mid-term Exam	S-1 to S-3		
7	L19	Syntax Analysis: Parser, Error-Recovery	S-4		
7	L20, L21	Syntax Analysis: left recursion, left factoring, Top-Down Parsers			
8	L22, L23	Syntax Analysis: FIRST and FOLLOW, Recursive-Descent and Predictive Parsing,	S-4	A-2	
8	L24	Syntax Analysis: LL(1) parsing, Review of Segment 4	S-4		
9	L25, L26	Parsing: Bottom-Up Parsing, Operator Precedence parsing, LR Parsers	S-5		
9	L27	Parsing: Construction of SLR	S-5		
10	L28, L29	Parsing: CLR, LALR Parse Tables, Shift-Reduce Parsing			
10	L30	Review of Segment 5 Class Test – 3	S-5	CT-3	
	Group-B (30 Marks)				
11	L31, L32	Syntax-Directed Translation: Syntax-Directed definitions, Semantic Rules, Dependency Graph, Attributes, Construction of Syntax trees	S-6		
11	L33	Syntax-Directed Translation: Type checking, Type expressions, Translation Scheme	S-6		
12	L34, L35	Syntax-Directed Translation: Type checking of expressions, statements and functions, Type conversions, statements and functions, Type conversions Review of Segment 6	S-6	A-3	
12	L36	Intermediate code generation: Run time environment, Activation tree, Control stack	S-7		

13	L37, L38	Intermediate code generation: Run time storage organization, Storage allocation strategies, Intermediate languages, three address statements, quadruples	S-7		
13	L39	Intermediate code generation: triples, indirect triples and polish notations Review of Segment 7	S-7	A-4	
14	L40, L41	Code Generation and Optimization: Code generation issues, Instruction costs, Basic blocks, Flow graphs, DAG representation, Next use information	S-8		
14	L42	Code Generation and Optimization: Transformations of Basic Blocks, Peephole optimization, Function preserving transformations	S-8		
15	L43, L44	Code Generation and Optimization: Optimization of Basic Blocks, Different models of code generation, error detection and error handling	S-8		
15	L45	Discussion Class for Final Exam Class test – 4	S-4 to S-8	CT-4	

Part C

19. Assessment Strategy:

The assessment and evaluation strategies for the course are given as follows:

Assessment Strategies	Description
Class Participation:	Students' individual in-class responses, attention, and sense of discipline, morality will be judged on the basis of 10 (ten) marks.
Class Test/Quiz:	Students will sit for only 1 (one) class test/quiz during the semester. The test/quiz will be taken before midterm. Class test/quiz marks will be assessed in 5 (five). No makeup class test will be taken. Students are strongly recommended not to miss any test.
Group Work:	The students will have to form groups consisting of a maximum of 4 members. There will be 1 assignment consisting of 5 (five) marks. The topics or case studies will be given as assignments in groups during the class which they have to prepare at home and will submit on or before the due date.
Assignment:	No late submission of assignments will be accepted.
Oral Presentation:	Students, in groups, will have to present the report of their assignments. Oral presentations of the students will be assessed in 5 (five) marks. No late presentation will be accepted.
Viva-vocé	Students will have to appear for viva-vocé before their Midterm (5 marks) and Final examination (5 marks).
Midterm Exam:	Midterm exam will be held according to the Academic Calendar published by the university. Midterm assessment marks will be 30 (thirty).
Final Exam:	Final exams will be held according to the Academic Calendar published by the university. Final assessment marks will be 40 (forty). Course contents learnt before Midterm examination will be included in the syllabus of Final Examination.
Make-up Procedure:	No late submission and/or make-up assignment/presentation/quiz will be allowed without prior permission and adequate and reasonable proof of absence.

20. Marks Distribution:

1. Course Assessment Pattern (Theory courses):

Bloom's Category		Evaluations out of 100 marks				
		CIE (50 marks)				SEE (50 marks)
Cognitive learning	Affective learning	Attendance Marks (10)	Assignment/ Lab Test (10)	Mid Lab (20)	External Participation in Curricular/Co-Curricular Activities (10)	Lab Exam (50)
Remember						

Understand					5	5
Apply			5	10		15
Analyze					5	
Evaluation			5	10		30
Create						
	Responding	10				
Total allocated marks		10	10	25	10	50

Note: CIE=Continuous Internal Evaluation, SEE= Semester End Examination

Delivery methods & activities: Lecture, White Board Writing, Questions and Answers, Discussions
Power point Presentation,

Assessment tools: Class Attendance, Class test, Quizzes/ Assignment on problem solution,
Mid-Term & Final Exam. Project evaluation & Viva

2. **Grading Policy:**As per IIUC grading policy

3. **Code of Conduct:**

- It is mandatory for all the students to participate in the class regularly and maintain proper discipline in the class.
- If a student fails to attend any class test, term exams, or final examination, he/she will get a zero in that class test, term, or final examination.
- Adopting unfair means in the exams will be considered as a serious crime and the student shall be placed to the university disciplinary committee.
- All the assignments, class test and exam copies should be neat and clear and demonstrate professionalism.
- No student is allowed to duplicate other student's work directly or with minor changes.
- Plagiarism is strictly restricted. One needs to provide a reference while using someone else's words, ideas, or research in assignments/exams.

Part D

21. Learning Materials:

Text Books:

#	Name of Authors	Title of Book	Edition	Publisher's Name	Year	ISBN
1.	A. V. Aho, R. Sethi, J.D. Ullman	Compiler Principle and Tools	2 nd	Pearson	2005	81-7808-046-X

Reference Books:

#	Name of Authors	Title of Book	Edition	Publisher's Name	Year	ISBN
1.	William A Barret, R. M. Bates	Compiler Construction theory and practice	2 nd		2007	
2.	Alfred V Aho, Monica S Lam, Ravi Sethi, Jeffrey D. Ullman	Compiler principles, technique and tools	2 nd	Pearson	2007	
3.	A.J Holub	Compiler Design in C			2007	

4.	Levine	<i>Flex and Bison: Text Processing Tools</i>		O'Reilly Media	2009	05961559 72
----	--------	--	--	-------------------	------	----------------